C1_Week3

Classification

Classification là một thể loại khác của Supervised Learning, mục đích của Classification là dự đoán một thể loại (Catergory) từ một input mới. Nếu với Linear Regression mình có thể có vô vàn prediciton, thì ở Classification chỉ có một số output hữu hạn duy nhất.

Motivations

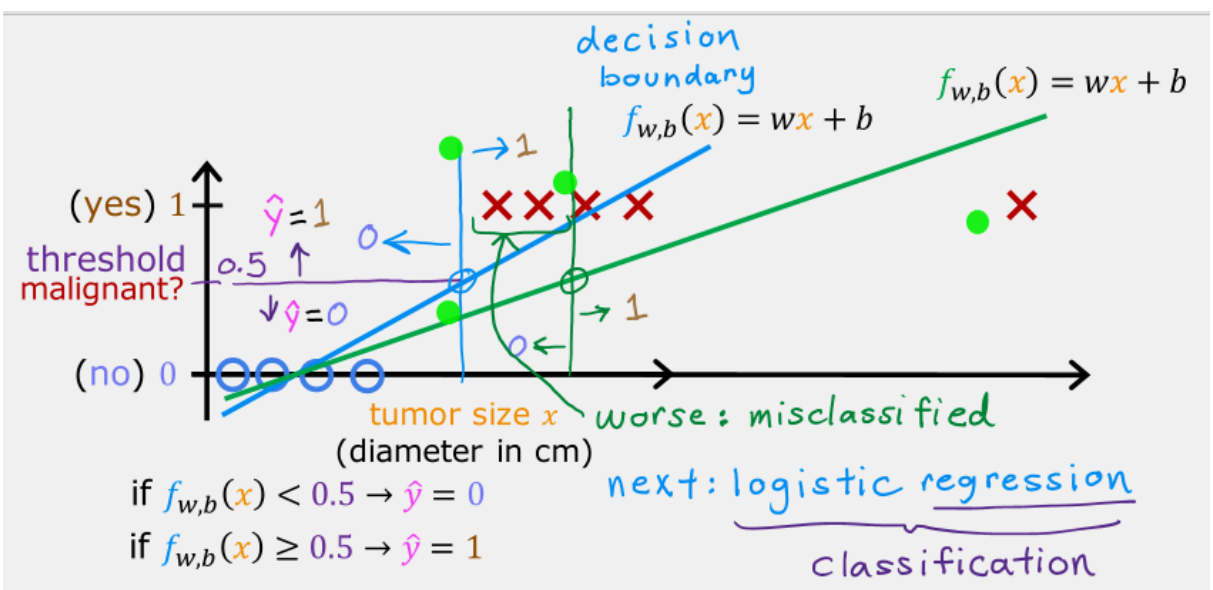
Các trường hợp cần dùng Classification:

- Mình cần phân loại xem liệu một khối u là lành tính hay ác tính
- Phân loại xem email có phải là spam hay không
- Liệu lần chuyển khoản này có đáng nghi không?

(Note nhỏ: các Classification mà chỉ có thể có 2 output duy nhất được gọi là một "Binary Classification")

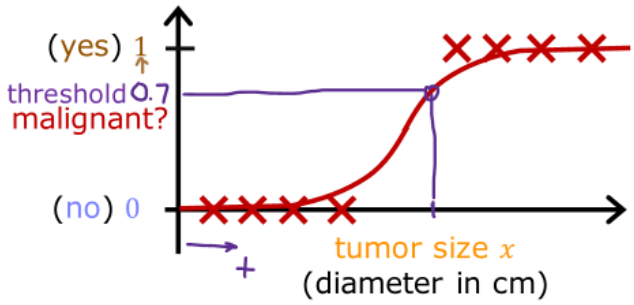
Linear Regression trong Classification

Trước khi đến với Logistic Regression, trong Classification mình cũng có thể dùng Linear Regression để phân loại output, chỉ là sẽ không ổn định, đặc biệt là khi có một Training Data nằm cá biệt. Dẫn tới đường thẳng của mình bị nghiêng quá mức, prediction dễ lệch.



Logistic Regression

Để có thể đưa ra các predicition theo nhóm/ lớp một cách ổn định, Logistic Regression sẽ sử dụng Sigmoid Function - phương trình luôn output ra xác suất từ 0 đến 1. Xác suất ấy sẽ là tỷ lệ % liệu input có thuộc một nhóm/ lớp ấy không.



- Logistics Regression có đường con nhìn như này, với phương trình được định nghĩa:

$$g(z) = \frac{1}{1 + e^{-z}}$$

- Model prediction của mình được viết dưới dạng:

$$f_{\vec{w},b}(\vec{x}) = g(z)$$

→ Sau đó mình có thể viết lại bằng tích vô hướng của vector x và w, bởi sẽ có nhiều feature, tương ứng với nhiều trọng số

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$$

→ Mặt khác, do hàm Sigmoid được viết ở dạng trên, mình có thể thay vào trong ngoặc:

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

$$e \approx 2.718$$

Như trên, về bản chất Model đang có tìm ra rằng "khả năng với input này, liệu có phải thuộc nhóm/ lớp này hay không", nên mình có thể biểu diễn theo ký hiệu này ngắn gọn:

$$f_{\vec{w},b}(\vec{x}) = P(y = 1 \mid \vec{x}; \vec{w}, b)$$

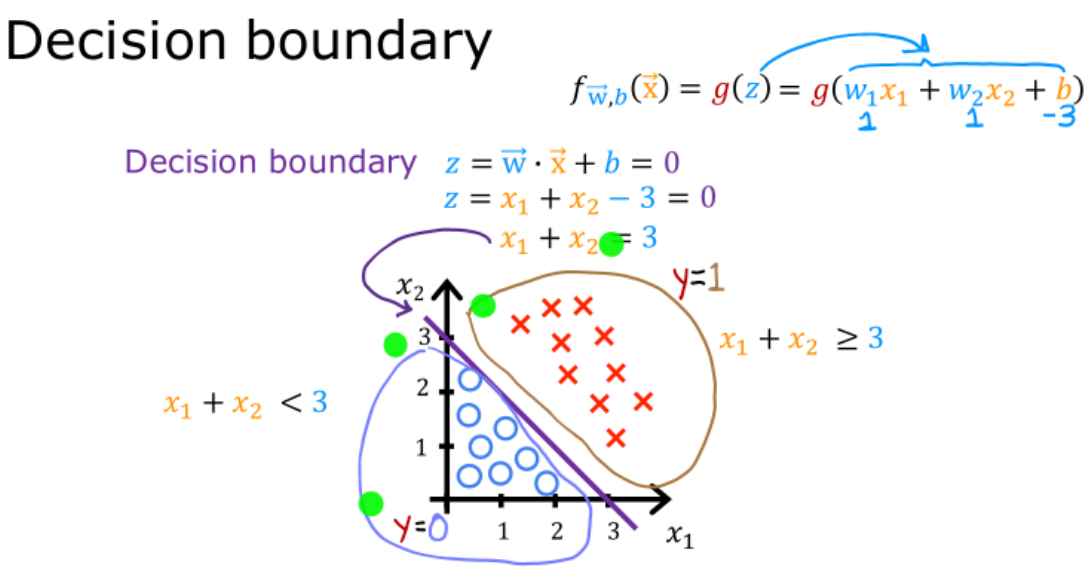
Do mình viết y = 1, nên đây là tỷ lệ % là “1”(ví dụ là khối u ác tính, email spam...), nên nếu thay y = 0 thì tổng của 2 tỷ lệ là 100%

$$P(y=0)+P(y=1)=1$$

Decision Boundary

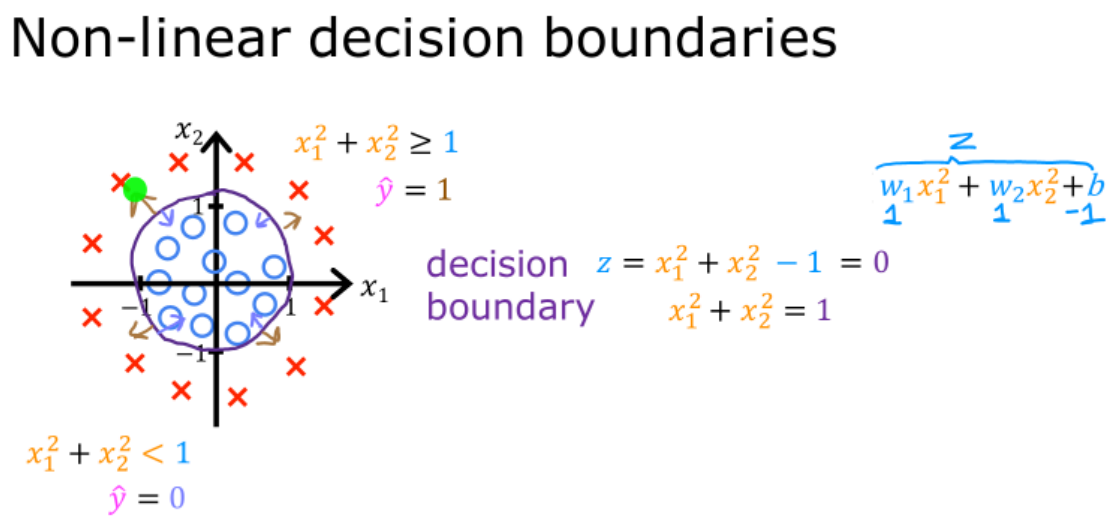
Ở bên trên mình đã có cách để tìm tỷ lệ giữa 0 và 1 (0% - 100%) của một input nhất định qua Sigmoid Function, nhưng làm thế nào để mình có thể quyết định được nên phân nhóm input đấy như nào? Mình sẽ sử dụng một Decision Boundary.

- Mình nênn hớ rằng Decision Boundary là một đường biên cố gắng chia cắt các vùng dự đoán của các nhóm/ lớp khác nhau. Tuy vậy mình có thể định nghĩa dựa trên một ngưỡng số, chẳng hạn như 0.5, nếu hàm Sigmoid ra một prediction >0.5, thì output là 1, <0.5 thì ngược lại.
- Đây là một số hình dạng của Đường Biên lựa chọn:
    - Decision Boundary là một đường thẳng, có thể hiểu là một đường thẳng trên mặt phẳng (x1,x2)

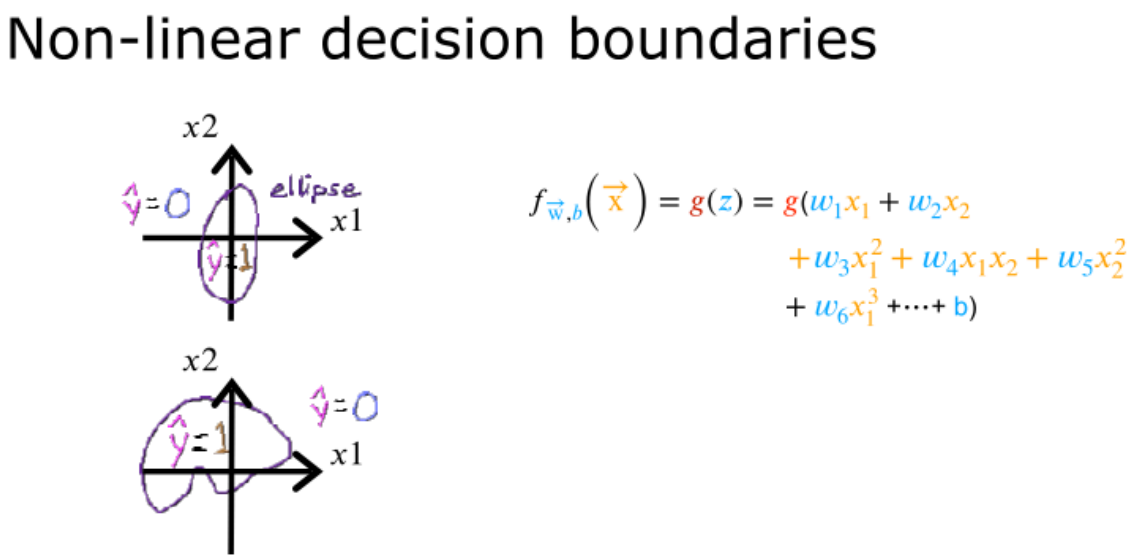


- Decision Boundary là một đường tròn.

https://loigiaihay.com/ly-thuyet-phuong-trinh-duong-tron-c45a5233.html

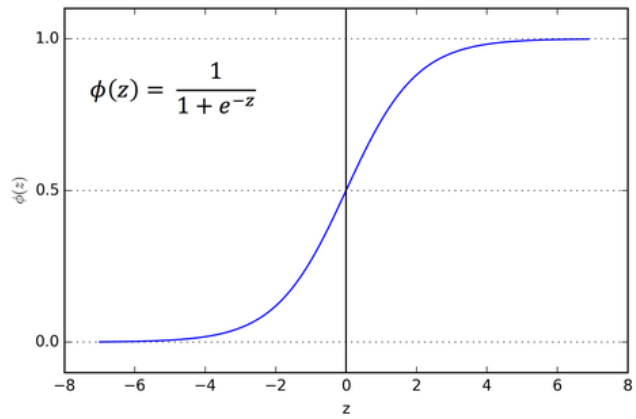


- Decision Boundary không có một hình dạng nhất định do là hàm đa thức



Note:

- Hình dạng của Decision Boundary phụ thuộc vào PHƯƠNG TRÌNH được đưa vào hàm Sigmoid, không phải là của CHÍNH hàm Sigmoid. Hàm Sigmoid luôn có đường cong như này:

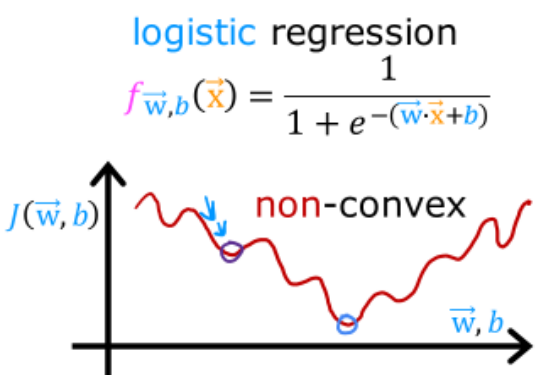


Cost Function For Logistic Regression

Nhớ lại phần Cost Function của Linear Regression, còn được gọi là Squared Error Cost Function:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2$$

Tuy vậy, để mình có thể áp dục được Cost Function, nhớ rằng Gradient Descent của mình phải tìm được cực tiểu (Global Minimum) của cả đồ thị, không chỉ là một Local Minimum. Khi mình sử dụng hàm Squared Error Cost Function sẽ vấp phải điều này.

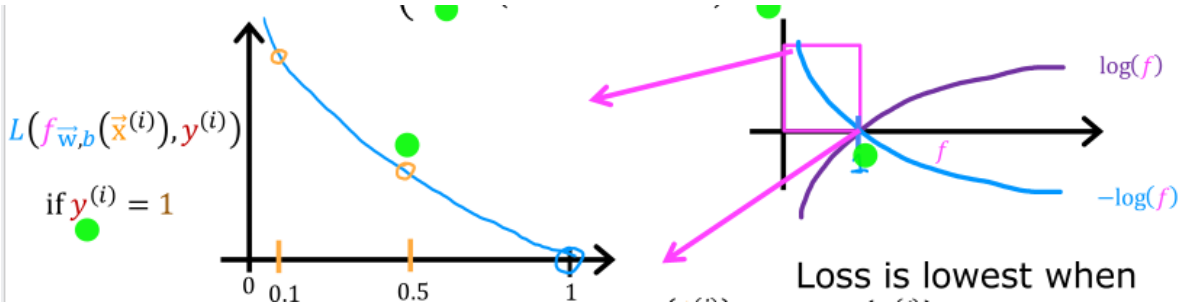


→ Mình cần tìm một Cost Function khác, gọi là Logistic Loss Function, được viết bằng:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}),\ y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 0 \end{cases}$$

- Hàm được viết dưới 2 trường hợp do có 2 target khả năng của một Model, là y = 0, y = 1. Do vậy.

- Do đầu vào của Logistic Cost Function là hàm Sigmoid, chỉ có thể đưa ra tỷ lệ giữa 0 và 1, nên khi biểu diễn trên đồ thị, 2 hàm Log âm dương nhìn như sau: (chỉ quan tâm đoạn [0,1])



Phần bên trái là phóng to của -log(f)

- Để giải thích lý do cho việc sử dụng hàm log, chẳng hạn khi target của mình là y = 1, việc Model của mình (hàm Sigmoid) predict ra một con số càng gần tới 0, Cost sẽ nhảy vọt lên rất rất cao, điều ngược lại cũng vậy. (tận dụng việc tiệm cận đứng của cả hàm Log là trục Oy)
- Một lý do khác là các hàm Log trong đoạn [0,1] có dạng convex, nên việc tìm Global Minimum qua Gradient Descent diễn ra dễ dàng.

Simplified Cost Function for Logistic Regression

Nhắc lại Logistic Cost Function:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}),\ y^{(i)}\right) = \begin{cases} -\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 1 \\ -\log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right) & \text{if } y^{(i)} = 0 \end{cases}$$

→ Mình có thể viết gọn lại thành phương trình trên một dòng, tận dụng 2 trường hợp y = 0 và y = 1 như sau:

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}),\ y^{(i)}\right) = -y^{(i)} \log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - \left(1 - y^{(i)}\right) \log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

- Hàm Cost Function khá dài, nhưng mình có thể chia thành 2 trường hợp:
- Nếu y = 1: (phép tính bên phải sẽ tự huỷ do nhân với 0)

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}),\ y^{(i)}\right) = -1 \cdot \log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - 0 \cdot \log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

- Nếu y = 0: (phép tính bên trái sẽ tự huỷ do nhân với 0)

$$L\left(f_{\vec{w},b}(\vec{x}^{(i)}),\ y^{(i)}\right) = -0 \cdot \log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) - 1 \cdot \log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)})\right)$$

- Kết hợp lại tất cả các công thức sẽ là một Logistic Cost Function hoàn chỉnh, bao gồm từng training data: (bỏ dấu trừ ra ngoài)

$$J(\vec{w},b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) + \left(1-y^{(i)}\right)\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right)\right]$$

Gradient Descent Implementation

Ta có công thức Logistic Cost Function:

$$J(\vec{w},b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(f_{\vec{w},b}(\vec{x}^{(i)})\right) + \left(1-y^{(i)}\right)\log\left(1-f_{\vec{w},b}(\vec{x}^{(i)})\right)\right]$$

- Gradient Descent sẽ chạy vòng lặp để update Weights và Bias như sau: (giống Linear Regression)

$$w_j = w_j - \alpha\frac{\partial}{\partial w_j}J(\vec{w},b)$$

$$b = b - \alpha\frac{\partial}{\partial b}J(\vec{w},b)$$

- Đạo hàm theo Weight để update Weight, đạo hàm theo Bias để update Bias:

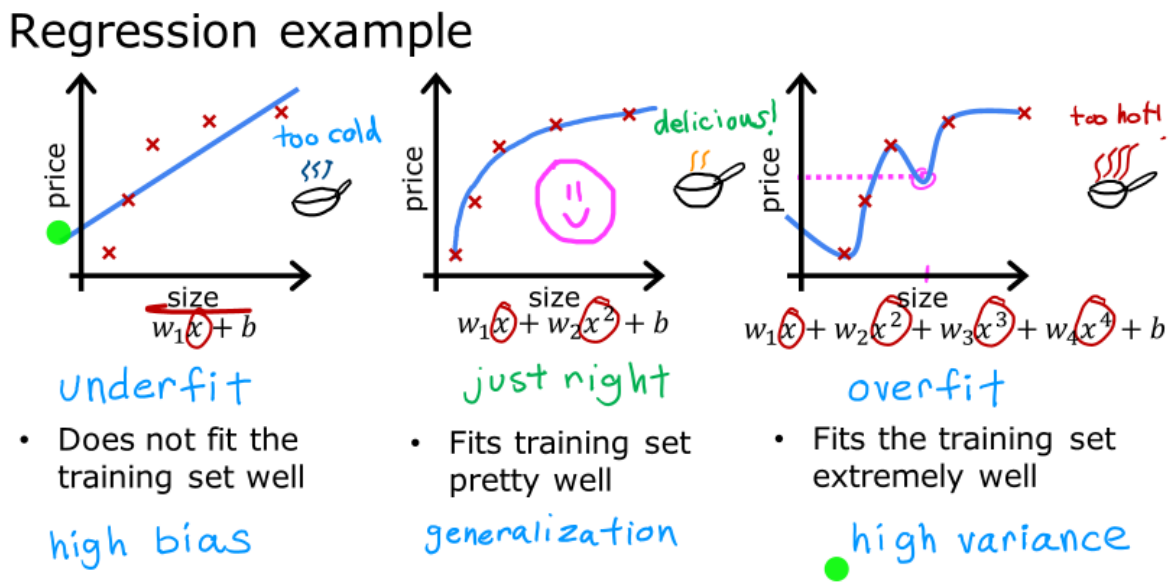
$$\frac{\partial}{\partial w_j}J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)x_j^{(i)}$$

$$\frac{\partial}{\partial b}J(\vec{w},b) = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}\right)$$

→ Một điều thú vị là cả phần đạo hàm và Gradient Descent của Logistic Regression giống với Linear Regression. Cùng chung format và đều phải Cập Nhật Đồng Thời.

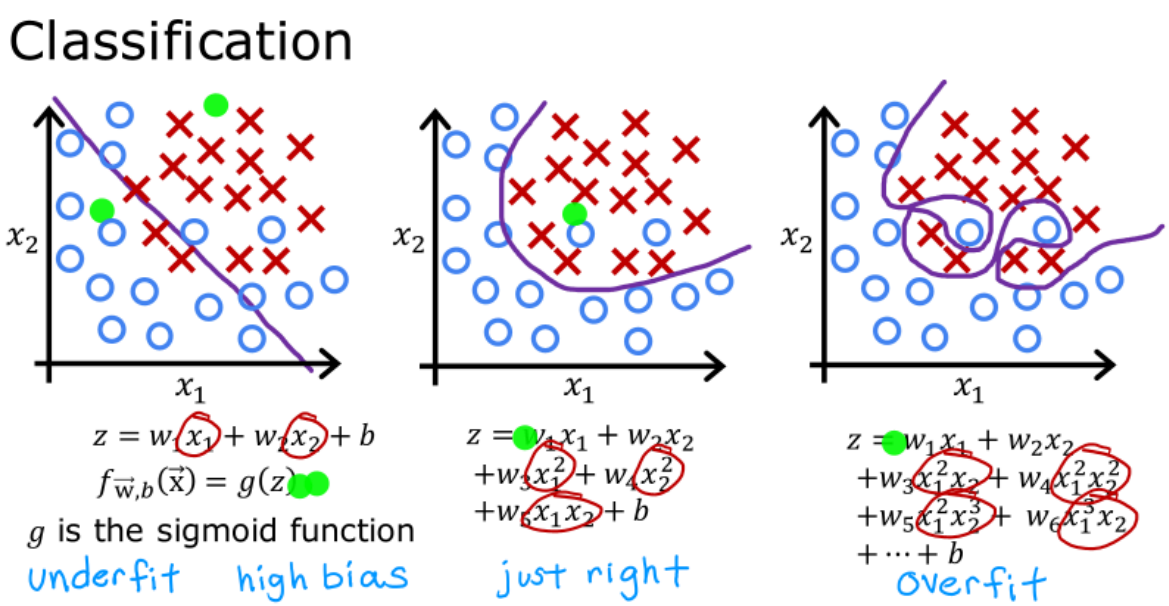
The Problem of Overfitting

Overfitting có nghĩa là "Vừa quá mức", việc một Model học quá kỹ dữ liệu đầu vào, cả những chi tiết không cần thiết, có một slide minh hoạ được cả 3 khái niệm Underfit, Fit, Overfit:



1. Underfit: không phù hợp, có nhiều sai sót với training data
2. Fit: phù hợp vừa đúng với training data
3. Overfit: "vừa quá mức" mà đưa ra dự đoán ở những chỗ không cần thiết, thường có các đa thức quá cao hoặc quá phức tạp

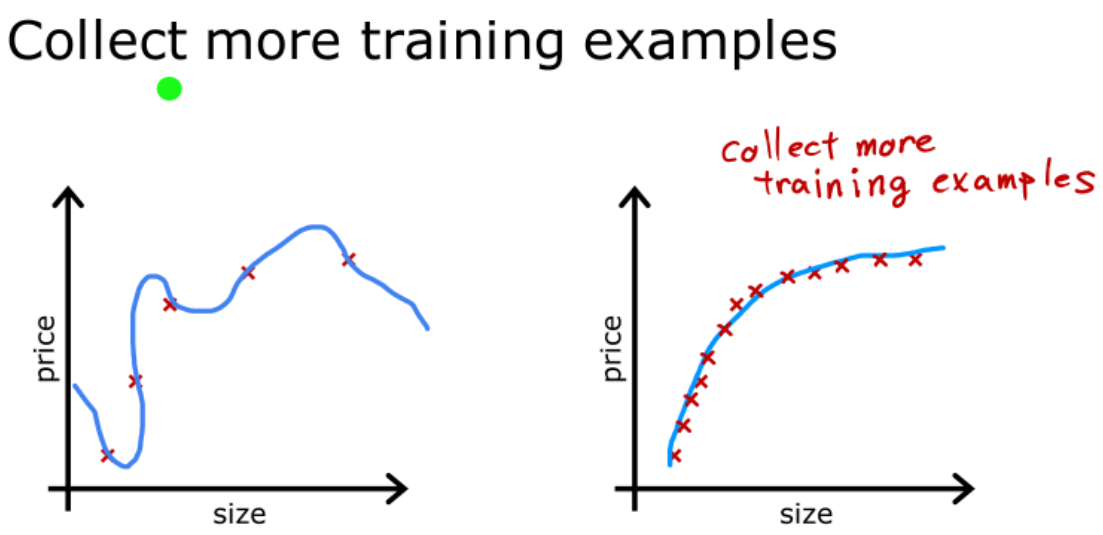
Đây là hình minh hoạ của cả 3 khái niệm, nhưng là của Classification:



Addressing Overfitting

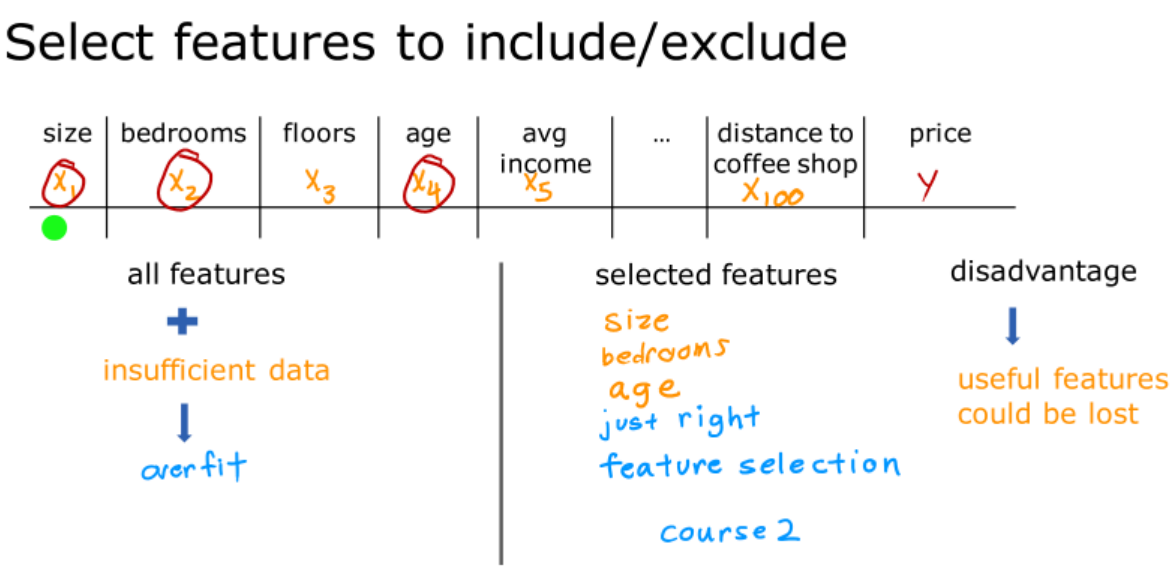
Đối với cách giải quyết Overfitting, mình có:

1. Tăng số lượng Training Data



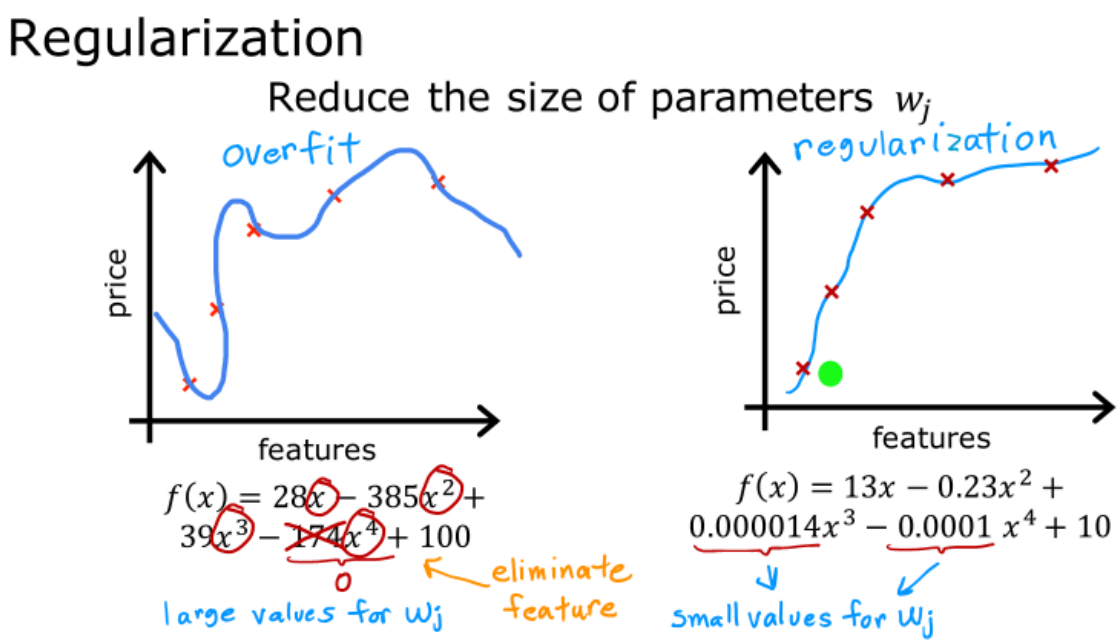
2. Lựa chọn nên/ không nên bao gồm các feature nhất định (đơn giản hoá số feature)

- Việc chọn lọc này có thể dẫn tới mất mát các feature hữu dụng nên cẩn thận.



3. Regularization

- Regularization sẽ được nói thêm ở dưới, trong phần này nhắc đến việc mình có thể giảm trọng số ở một số feature nhất định, đặc biệt là ở feature mang đa thức bậc cao.



Cost Function with Regularization

Khi nhắc qua về Regularization ở bên trên về việc giảm trọng số ở một số feature nhất định (các feature mang đa thức bậc cao), mình có thể viết thêm một phần vào Cost Function để cho khi chạy Gradient Descent, các trọng số ấy sẽ tự giảm xuống bởi việc Cost tăng cao.

→ Bổ sung thêm vào Cost Function như sau:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 + \frac{\lambda}{2m} b^2$$

Trên thực tế việc mình điều chỉnh bias là không cần thiết nên có thể được rút ngắn lại thành:

$$\min_{\vec{w},b} J(\vec{w}, b) = \min_{\vec{w},b} \left[\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \right]$$

- Lambda trong công thức có thể được tự chọn là một giá trị nào đó, trong slide ghi là 10^10

Regularized Linear Regression

Hàm Cost Function được viết lại ở bên trên, giờ với cả Regularization:

$$\min_{\vec{w},b} J(\vec{w}, b) = \min_{\vec{w},b} \left[\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \right]$$

- Gradient Descent giống như trước sẽ lặp đi lặp lại các update sau:

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

→ Với đạo hàm theo các phần tương ứng: (giờ có thêm phần Weight ở đằng sau)

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$

- Việc thêm phần tính trọng số đằng sau của đạo hàm là để giúp khi nhân với stepping size, vế phải sẽ lớn hơn một chút, dẫ đến việc w khi cập nhật sẽ nhỏ hơn bình thường (mục đích của regularization)

▼ Chứng minh công thức đạo hàm (xem tham khảo)

Ta có:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \right]$$

Với:

$$f_{\vec{w},b}(\vec{x}^{(i)}) = \vec{w} \cdot \vec{x}^{(i)} + b$$

Ta viết lại:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{2m} \left[\sum_{i=1}^{m} 2 \left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{2m} \cdot 2w_j \cdot 2m \right]$$

Rút gọn số 2:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Rút ngắn lại thành ký hiệu phương trình:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Ta có:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \right]$$

Với:

$$f_{\vec{w},b}(\vec{x}^{(i)}) = \vec{w} \cdot \vec{x}^{(i)} + b$$

Ta viết lại:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{2m} \left[\sum_{i=1}^{m} 2 \left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{2m} \cdot 2w_j \cdot 2m \right]$$

Rút gọn số 2:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Rút ngắn lại thành ký hiệu phương trình:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

Regularized Logistic Regression

Phần này giống bên trên chỉ là khác Model.

Khi mình Regularize Logistic Regression mình chỉ thêm phần của trọng số vào sau như phần trên:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log\left(f_{\vec{w},b}(\vec{x}^{(i)}) \right) + \left(1 - y^{(i)} \right) \log\left(1 - f_{\vec{w},b}(\vec{x}^{(i)}) \right) \right] + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Công thức của Gradient Descent: (như trên)

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

Với đạo hàm được tính bằng:

$$\frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$\frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)$$